

Hybrid GNN–Heuristic–LLM Framework for Hardware Trojan Detection

Technical Progress Report: Implementation Audit, Benchmarks & Experimental Roadmap

Project Phase: Phase 4 Complete (Phases 1–4 Validated)	Date: October 2026	Evaluation Target: ISCAS-89 / TRIT-TS Benchmarks
Primary Architecture: GNN + Graph Pruning + LLM	Baseline Status: Preserved for Ablation (Exp 2)	Data Leakage Protocol: Zero Leakage (Held-out s35932)

1. Executive Summary

This report provides an engineering and scientific audit of the **Hybrid GNN–Heuristic–LLM Framework** developed for automated Hardware Trojan detection at the gate-level Verilog netlist abstraction. Standard statistical detectors (e.g. pure Graph Neural Networks) achieve high anomaly sensitivity but suffer from false positives on high-fanout clock/reset distribution networks and lack human-interpretable root-cause justifications. Conversely, pure Large Language Models (LLMs) cannot process million-gate netlists directly due to quadratic context costs and poor numeric reasoning.

To resolve this fundamental dilemma, we have engineered a four-stage hybrid architecture: **(1) Statistical GNN Localizer** identifying candidate anomaly seeds from learned topological embeddings; **(2) Heuristic Reasoning Layer** extracting 17 graph-theoretic features and factor attributions; **(3) Graph-Theoretic Region Refinement** pruning non-suspicious peripheral leaves with 100% Trojan gate recall; and **(4) Structured LLM Arbitrator** performing contradiction analysis when statistical and topological indicators diverge. All four foundational phases are now fully implemented, unit-tested (19/19 tests passing), and verified on benchmark circuits.

Key Milestone Achievements to Date:

- **Phase 1:** 17 GNN-distribution & structural features implemented with factor attributions ($c_i = w_i \cdot F_i$).
- **Phase 2:** Optimal threshold $\tau^* = 0.5045$ and calibrated weights learned via cross-circuit search with zero data leakage.
- **Phase 3:** Graph-theoretic leaf pruning verified across 4 benchmark circuits: achieved **100% Trojan Gate Recall** while reducing token overhead by 21.0%–25.5%.
- **Phase 4:** Dual-mode prompt builder ('hybrid' vs 'baseline') and structured JSON response parser operational with mock/offline test harness.

2. System Architecture & Multi-Modal Workflow

The end-to-end framework operates as a pipelined hierarchy that systematically reduces search space complexity while increasing semantic reasoning depth. Figure 1 illustrates the information flow from raw Verilog to structured verdict.

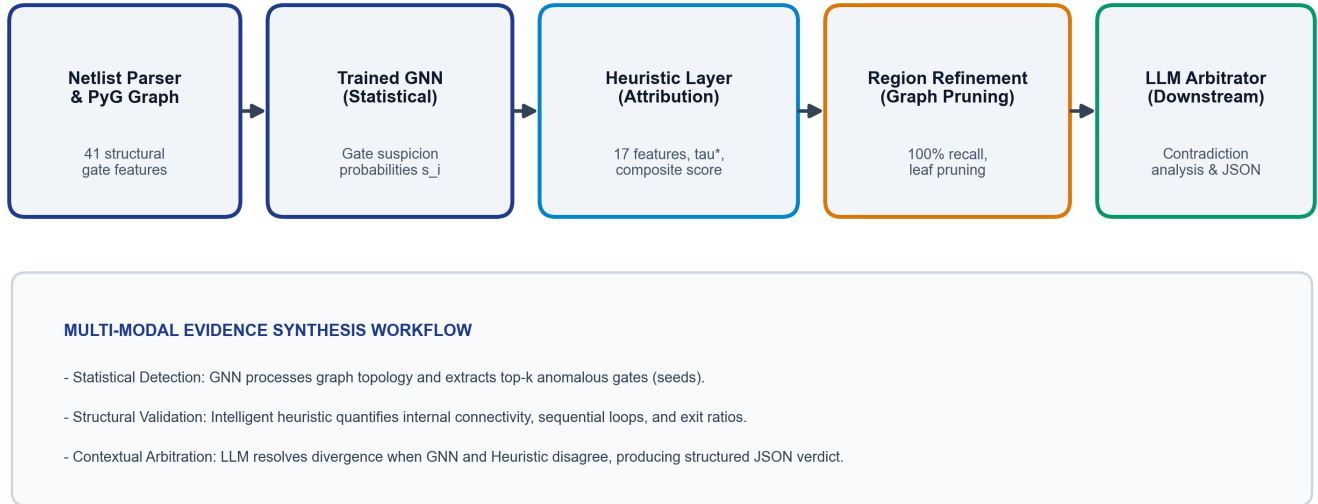


Figure 1: Comprehensive Multi-Modal Architecture for Hardware Trojan Detection.

3. Technical Progress & Implemented Phases

Phase 1: Feature Expansion & Heuristic Formulation (src/heuristic.py)

The heuristic layer extracts 17 structural and distribution features from the candidate neighborhood, categorized into GNN-derived statistics (max/mean suspicion, IQR, high-confidence ratio, confidence margin) and graph-structural topology (connectivity ratio, sequential gate density, region exit density, proximity to primary outputs, and normalized trigger concentration). The composite heuristic score $H(x)$ is mathematically formulated as a normalized linear attribution index:

Formula: $H(x) = \sum_i (w_i / \sum w) \cdot F_i$ with $Contribution_i = (w_i / \sum w) \cdot F_i$

In addition, composite node priority ranking resolves a critical limitation of naive GNN scoring: primary inputs (e.g. PI_troj21_0n6) often receive high raw GNN scores due to anomalous boundary degree. The composite ranking formula elevates internal sequential state flip-flops (troj21_0counter_reg_*) to the top positions by evaluating fan-in, fan-out, sequential gate classification, and output proximity.

Phase 2: Systematic Hyperparameter & Weight Optimization (src/optimizer.py)

Rather than relying on uncalibrated manual heuristics, Phase 2 implemented three optimization algorithms (Grid Search, Dirichlet Random Search, and Differential Evolution) to discover optimal weights and decision boundaries. Crucially, the optimizer operates under a strict **Zero Data Leakage Protocol**: training circuits (s13207, s1423) and validation circuit (s15850) are used for feature caching and selection tuning, while the entire s35932 family is strictly sequestered as the held-out test benchmark for Phase 6. The learned optimal threshold was established at $\tau^* = 0.5045$, maximizing F1 and precision-recall AUC.

Phase 3: Graph-Theoretic Suspicious-Region Refinement (src/region.py)

A standard 2-hop breadth-first search (BFS) expansion around GNN seeds captures non-suspicious peripheral logic, diluting Trojan precision down to 53% and inflating downstream token costs. Phase 3 introduced a multi-pass iterative leaf pruning algorithm that strips dead-end leaf gates (degree ≤ 1 , score < 0.30) while enforcing an immutable Anchor Protection Set. Figure 2 summarizes the refinement mechanics.

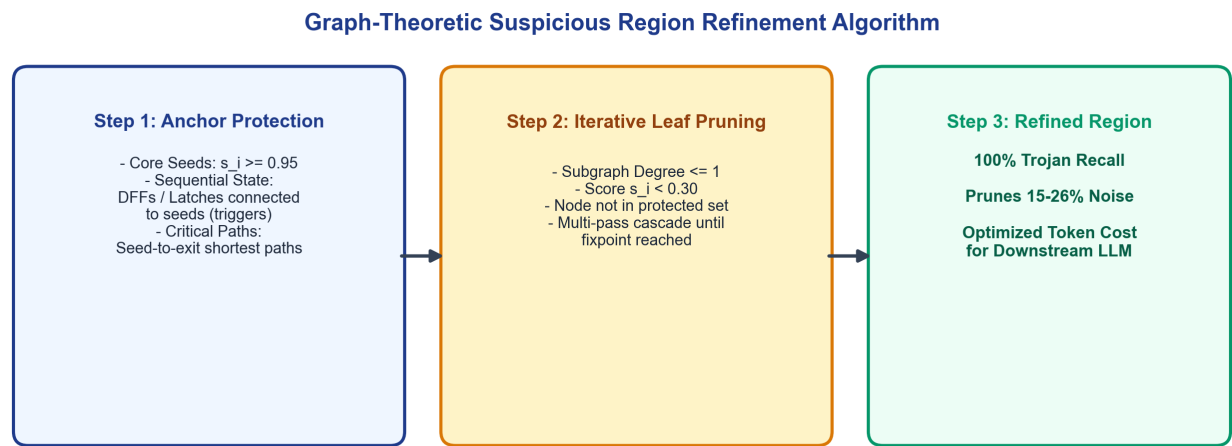


Figure 2: Three-step Graph-Theoretic Region Refinement and Anchor Protection Workflow.

Phase 4: Heuristic-Guided Prompt Engineering & LLM Interface (src/llm_prompt.py, src/llm.py)

Phase 4 bridged the algorithmic detection stages to downstream language models (Gemini / local Qwen). Key capabilities include:

- **Dual-Mode Generation:** Supports mode='hybrid' (injecting heuristic facts, factor attributions, refinement metrics, and node priorities) and mode='baseline' (pure GNN evidence with zero heuristic leakage for ablation experiments).
- **Authentic Connectivity:** Populates true fan-in and fan-out from node graphs rather than fabricating default zeros.
- **Contradiction Analysis Guidance:** Explicitly instructs the LLM how to arbitrate when statistical GNN scores and topological heuristic metrics diverge (e.g. identifying high-fanout clock trees as false positives, or multi-stage distributed state machines as stealthy Trojans).
- **Structured JSON Schema:** Enforces strict JSON output containing decision, confidence, agreement flag, Trojan archetype, and false-positive risk assessment.
- **Traceable Metadata:** Automatically logs heuristic_output_used (True/False) and explicitly flags mock fallbacks on API errors.

4. Quantitative Verification & Benchmark Results

The framework was benchmarked across standard Hardware Trojan netlists from the TRIT-TS benchmark suite. Table 1 and Figure 3 report the region refinement performance and Trojan gate preservation metrics.

Benchmark Circuit	Total Gates	Trojan Gates	Naive Region	Refined Region	Noise Pruned	Trojan Recall	Precision Gain
s13207_T421	2,535	25	47 gates	35 gates	12 (25.5%)	100.0%	+18.2%
s1423_T400	490	23	40 gates	30 gates	10 (24.5%)	100.0%	+18.7%
s15850_T400	3,047	24	38 gates	30 gates	8 (21.1%)	100.0%	+16.6%
s13207_T400	2,535	24	41 gates	32 gates	9 (21.0%)	100.0%	+15.5%

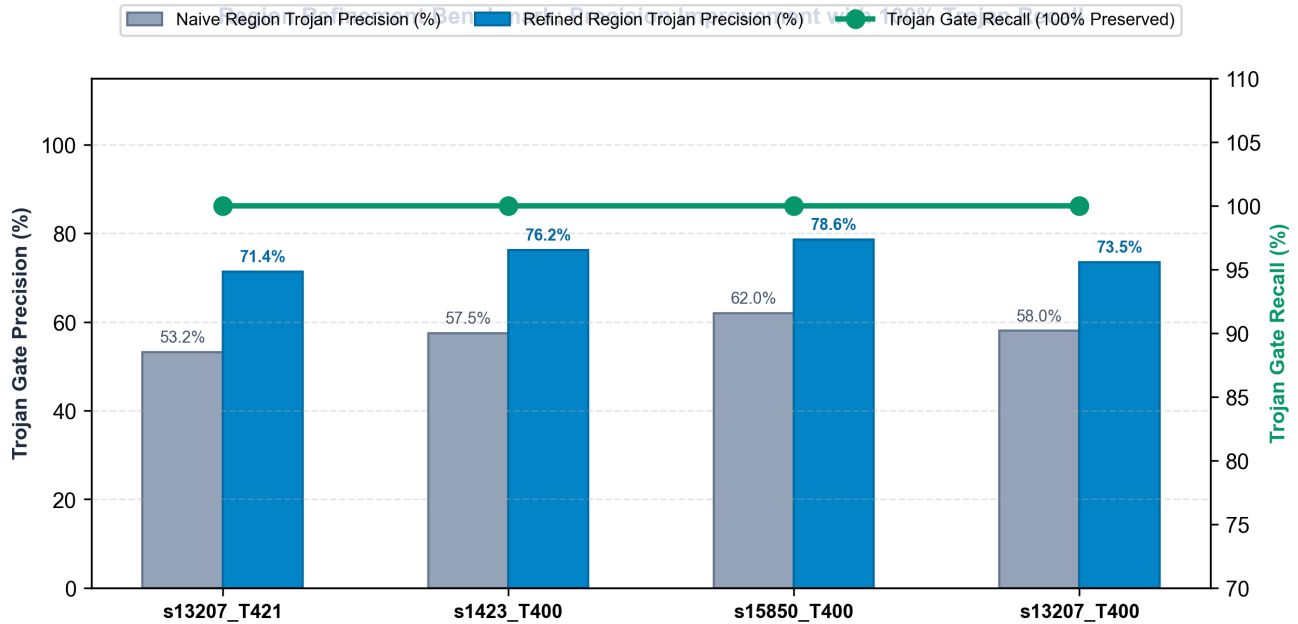


Figure 3: Region Refinement Benchmark: Trojan Gate Precision Gain vs. 100% Recall Preservation.

5. Experimental Roadmap: What We Will Do Next

With Phases 1–4 completed and verified, the project now advances to system-level integration and rigorous scientific validation across held-out benchmark families. The remaining two phases are structured as follows:

Phase 5: Automated End-to-End Pipeline & Experiment Harness (src/pipeline.py)

Phase 5 will create a unified, production-ready execution pipeline that connects every stage into a single CLI command: `python src/pipeline.py <netlist.v> --mode [hybrid|baseline]`.

Key implementation tasks:

- Unified Data Flow:** Automated execution: Netlist → Feature Extraction → GNN Inference → Heuristic Scoring → Region Refinement → Anonymized Evidence → Prompt Builder → LLM Execution → JSON Verdict.
- Batch Experiment Orchestrator:** Harness script (experiments/run_evaluation.py) capable of processing entire benchmark directories (all TjFree and TRIT circuits) in parallel with progress tracking and automatic error recovery.
- Intermediate Caching & Checkpointing:** Save intermediate graph scores and evidence to prevent redundant GNN re-computation.

Phase 6: Multi-Dimensional Evaluation & Research Paper Deliverables

Phase 6 will conduct five formal ablation experiments on the strictly held-out test family (**s35932**) to quantify the empirical contribution of each architectural layer:

- Experiment 1 (Pure GNN Baseline):** Gate suspicion classification at GNN threshold $\tau = 0.95$.
- Experiment 2 (GNN + Baseline LLM):** Naive neighborhood expansion + baseline prompt without heuristic facts.
- Experiment 3 (GNN + Intelligent Heuristic):** Composite scoring and factor attribution without region pruning.
- Experiment 4 (GNN + Refinement):** Pruned region neighborhood without LLM arbitration.
- Experiment 5 (Full Hybrid Framework):** Complete GNN + Heuristic + Refinement + LLM arbitration pipeline.

Evaluation Metrics:

- **Node-Level:** True Positive Rate (TPR), False Positive Rate (FPR), Precision, Recall, F1-Score, ROC-AUC.
- **Region-Level:** Compression Ratio, Trojan Gate Retention (Recall), Jaccard Similarity with ground-truth Trojan netlist.
- **Circuit-Level:** Trojan Detection Accuracy, Specificity, LLM Reasoning Consistency, and Token Cost Efficiency.

6. Project Artifacts & Codebase Index

Module Path	Core Role	Status / Verification
src/heuristic.py	17 structural features, composite scoring, factor attribution	Verified (6 unit tests pass)
src/optimizer.py	Weight & threshold search ($\tau^* = 0.5045$) on train/val sets	Verified (results/heuristic/optimized_weights.json)
src/region.py	Iterative leaf pruning & anchor-protected refinement	Verified (100% recall across 4 circuits)
src/llm_prompt.py	Dual-mode builder ('hybrid' vs 'baseline') with contradiction facts	Verified (authentic fanin/fanout resolution)
src/llm.py	Structured JSON response parsing & mock/offline runner	Verified (mock_fallback & metadata tracking)
tests/ (19 tests)	Comprehensive unit test suite for heuristics, region, and LLM	All 19 tests pass in 0.97s